

ONEDRAFT

CAD-AI — Aria Powered

Artifact & Storage Management Engine

Version 0.1

Status: Implementation Specification

Storage Model: Object Storage + PostgreSQL Metadata

API: REST / JSON

API Version: /api/v1

Identifiers: UUID

Primary Model: Immutable Versioned Artifacts

AI Layer: Aria Powered

1. ARTIFACT ENGINE DESIGN PRINCIPLE

The OneDraft Artifact & Storage Management Engine provides the persistent storage boundary for every generated, uploaded, imported, rendered, exported, and finalized project artifact.

The system shall distinguish between:

Project state

The computational architectural model stored by OneDraft.

Artifact metadata

The database record describing a stored artifact.

Artifact content

The actual binary or textual object stored in object storage.

Artifact version

A specific immutable content state.

Artifact derivative

A generated representation derived from another artifact.

Artifact package

A controlled collection of artifacts representing a project deliverable.

Artifact provenance

The complete chain identifying how an artifact was created.

The artifact engine therefore becomes the persistent bridge between the computational project model and physical deliverables.

2. STORAGE ARCHITECTURE

The storage architecture shall initially consist of:

```
OneDraft Application
    ↓
Artifact Service
    ↓
PostgreSQL Metadata
    ↓
Object Storage
    ↓
Artifact Content
```

PostgreSQL stores metadata, relationships, hashes, ownership, version information and provenance.

Object storage stores the actual artifact payload.

The application shall never treat an object-storage path alone as the authoritative identity of an artifact.

3. STORAGE SCHEMAS

The PostgreSQL database shall initially use:

```
storage
artifacts
artifact_versions
artifact_derivatives
artifact_links
artifact_packages
artifact_package_items
uploads
downloads
provenance
```

The existing OneDraft logical schemas remain authoritative for project, model, documentation, compliance, validation, revision and audit data.

4. UNIVERSAL ARTIFACT RULES

All artifact identifiers use:

UUID

All timestamps use:

TIMESTAMPTZ

All timestamps are stored in UTC.

Artifacts are immutable after publication.

A new content state creates a new artifact version rather than overwriting an existing version.

Normal application operation must never silently replace historical artifact content.

5. ARTIFACT TABLE

storage.artifacts

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
artifact_type VARCHAR(64) NOT NULL
name VARCHAR(512) NOT NULL
description TEXT
mime_type VARCHAR(255)
file_extension VARCHAR(32)
status VARCHAR(32) NOT NULL
current_version_id UUID
created_by UUID
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

Indexes:

```
idx_artifacts_project
idx_artifacts_type
idx_artifacts_status
idx_artifacts_updated
```

The artifact record identifies the logical object.

The artifact version identifies its immutable content state.

6. ARTIFACT TYPES

Initial artifact types include:

```
SOURCE_DOCUMENT
PROJECT_SPECIFICATION
```

SURVEY
REFERENCE_IMAGE
REFERENCE_DRAWING
CAD_FILE
GEOMETRY
MODEL_EXPORT
VIEW
DRAWING
SHEET
SCHEDULE
REPORT
VALIDATION_REPORT
CODE_SOURCE
CODE_MANIFEST
REDLINE
RENDER
PDF
DOCUMENT_PACKAGE
ARCHIVE

Future artifact types may be added without changing the fundamental storage model.

7. ARTIFACT VERSIONS

`storage.artifact_versions`

Fields:

id UUID PRIMARY KEY
artifact_id UUID NOT NULL
version_number INTEGER NOT NULL
storage_provider VARCHAR(64) NOT NULL
bucket VARCHAR(255) NOT NULL
object_key TEXT NOT NULL
byte_size BIGINT NOT NULL
mime_type VARCHAR(255)
content_hash VARCHAR(128) NOT NULL
etag VARCHAR(255)
compression VARCHAR(64)
encryption_status VARCHAR(32)
created_by UUID
created_at TIMESTAMPTZ NOT NULL

Unique:

`UNIQUE(artifact_id, version_number)`

Unique content protection:

`UNIQUE(content_hash)`

where appropriate to the deployment architecture.

8. CONTENT HASH

Every immutable artifact version shall receive a cryptographic content hash.

The initial implementation shall support:

SHA-256

The hash is calculated from the stored content.

Conceptually:

```
CONTENT
↓
SHA-256
↓
CONTENT_HASH
↓
ARTIFACT_VERSION
```

The content hash provides deterministic identity verification.

9. STORAGE OBJECT KEY

Object storage keys shall not expose uncontrolled user-provided filenames as authoritative storage paths.

The initial object-key structure shall be conceptually:

```
projects/{project_id}/artifacts/{artifact_id}/versions/{version_id}/content
```

Optional derivatives:

```
projects/{project_id}/artifacts/{artifact_id}/versions/{version_id}/derivatives/
{derivative_id}/content
```

This prevents filename collisions and preserves deterministic storage organization.

10. OBJECT STORAGE ABSTRACTION

The application shall communicate with object storage through a storage abstraction.

Conceptually:

```
StorageProvider
  put()
  get()
  delete()
  exists()
```

```
head()  
generate_upload_url()  
generate_download_url()
```

The domain layer shall not depend directly on a specific cloud storage vendor.

11. STORAGE PROVIDERS

The initial implementation shall support an S3-compatible storage interface.

Possible deployments include:

```
AWS S3  
MinIO  
Cloud-compatible S3 storage  
Self-hosted object storage
```

The provider is selected through configuration.

The OneDraft domain model remains provider-neutral.

12. UPLOAD RECORDS

`storage.uploads`

Fields:

```
id UUID PRIMARY KEY  
project_id UUID NOT NULL  
artifact_id UUID  
requested_by UUID NOT NULL  
filename VARCHAR(512) NOT NULL  
mime_type VARCHAR(255)  
expected_size BIGINT  
upload_method VARCHAR(64) NOT NULL  
status VARCHAR(32) NOT NULL  
storage_key TEXT  
content_hash VARCHAR(128)  
created_at TIMESTAMPTZ NOT NULL  
completed_at TIMESTAMPTZ
```

Initial upload statuses:

```
CREATED  
UPLOADING  
UPLOADED  
VERIFYING  
COMPLETED  
FAILED  
CANCELLED
```

13. DIRECT UPLOAD MODEL

Large files should not pass unnecessarily through the OneDraft API service.

The preferred workflow is:

```
CLIENT
↓
REQUEST UPLOAD
↓
ARTIFACT SERVICE
↓
SIGNED UPLOAD URL
↓
OBJECT STORAGE
↓
UPLOAD COMPLETE
↓
ARTIFACT SERVICE
↓
HASH VERIFICATION
↓
ARTIFACT VERSION
```

This prevents the API layer from becoming a large-file transport bottleneck.

14. UPLOAD INITIALIZATION

Endpoint:

POST /api/v1/projects/{project_id}/artifacts/uploads

Request:

```
{
  "filename": "survey.pdf",
  "mime_type": "application/pdf",
  "size": 18420342,
  "artifact_type": "SURVEY"
}
```

Response:

```
{
  "data": {
    "upload_id": "uuid",
    "artifact_id": "uuid",
    "upload_url": "signed-url",
    "expires_at": "ISO-8601"
  }
}
```

The upload URL is temporary and must not become the permanent artifact identity.

15. UPLOAD COMPLETION

Endpoint:

POST /api/v1/projects/{project_id}/artifacts/uploads/{upload_id}/complete

Request:

```
{
  "content_hash": "sha256",
  "size": 18420342
}
```

The server verifies:

- upload exists
- project authorization
- object exists
- size matches
- content hash matches

Only after verification is the artifact version marked complete.

16. ARTIFACT RETRIEVAL

Endpoint:

GET /api/v1/projects/{project_id}/artifacts/{artifact_id}

Response:

```
{
  "data": {
    "artifact_id": "uuid",
    "name": "survey.pdf",
    "type": "SURVEY",
    "status": "ACTIVE",
    "current_version_id": "uuid",
    "mime_type": "application/pdf"
  }
}
```

17. ARTIFACT DOWNLOAD

Endpoint:

GET /api/v1/projects/{project_id}/artifacts/{artifact_id}/download

The API returns a temporary signed download URL.

The artifact service remains responsible for authorization.

Object storage credentials are never exposed to the client.

18. ARTIFACT DERIVATIVES

storage.artifact_derivatives

Fields:

```
id UUID PRIMARY KEY
source_artifact_version_id UUID NOT NULL
artifact_id UUID NOT NULL
derivative_type VARCHAR(64) NOT NULL
version_id UUID NOT NULL
generation_job_id UUID
parameters JSONB
created_at TIMESTAMPTZ NOT NULL
```

Examples:

```
PDF → thumbnail
CAD → preview
drawing → raster preview
document → text extraction
model → lightweight viewer representation
PDF → searchable text
```

19. DERIVATIVE PROVENANCE

Every derivative shall identify its source.

Conceptually:

```
SOURCE ARTIFACT
  ↓
SOURCE VERSION
  ↓
GENERATION PARAMETERS
  ↓
GENERATION JOB
```

↓
DERIVATIVE

A derivative must never be presented as an independent source artifact.

20. ARTIFACT LINKS

storage.artifact_links

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
source_artifact_id UUID NOT NULL
target_artifact_id UUID NOT NULL
relationship_type VARCHAR(64) NOT NULL
metadata JSONB
created_at TIMESTAMPTZ NOT NULL
```

Initial relationships:

```
SOURCE_OF
DERIVED_FROM
ATTACHED_TO
REFERENCES
GENERATED_FROM
VALIDATED_BY
REPRESENTS
SUPERSEDES
```

This extends the OneDraft dependency graph into physical artifacts.

21. PROJECT ARTIFACT RELATIONSHIP

A project may contain:

```
SOURCE DOCUMENTS
↓
PROJECT REQUIREMENTS
↓
PROJECT MODEL
↓
GEOMETRY
↓
VIEWS
↓
DRAWINGS
↓
SHEETS
↓
```

DOCUMENT PACKAGE

The artifact engine shall preserve this relationship without duplicating domain ownership.

22. DOCUMENT PACKAGE STORAGE

The existing:

`documentation.document_packages`

record remains the authoritative document-package domain record.

The artifact engine stores the physical package.

The relationship becomes:

Document Package

↓

Artifact

↓

Artifact Version

↓

Object Storage

23. PACKAGE CONTENT

`storage.artifact_packages`

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
document_package_id UUID NOT NULL
artifact_id UUID NOT NULL
package_type VARCHAR(64) NOT NULL
manifest JSONB NOT NULL
package_hash VARCHAR(128)
status VARCHAR(32) NOT NULL
created_at TIMESTAMPTZ NOT NULL
```

The package manifest identifies every included artifact version.

24. PACKAGE ITEMS

`storage.artifact_package_items`

Fields:

```
package_id UUID NOT NULL
artifact_id UUID NOT NULL
artifact_version_id UUID NOT NULL
sequence INTEGER NOT NULL
relative_path TEXT NOT NULL
role VARCHAR(64)
required BOOLEAN DEFAULT TRUE
content_hash VARCHAR(128) NOT NULL
```

Primary key:

(package_id, artifact_version_id)

This makes a document package reproducible.

25. PACKAGE MANIFEST

Every finalized package shall contain a machine-readable manifest.

Conceptually:

```
{
  "project_id": "uuid",
  "revision_id": "uuid",
  "model_version_id": "uuid",
  "code_manifest_id": "uuid",
  "validation_run_id": "uuid",
  "artifacts": [
    {
      "artifact_id": "uuid",
      "artifact_version_id": "uuid",
      "path": "A101.pdf",
      "sha256": "..."
    }
  ]
}
```

The manifest becomes part of the package provenance.

26. ARTIFACT PROVENANCE

The provenance subsystem shall record:

```
source
operation
input artifacts
input versions
project
model version
```

```
revision
code manifest
validation run
generation job
actor
timestamp
output artifact
```

This creates the artifact lineage graph.

27. PROVENANCE RECORD

provenance.artifact_provenance

Fields:

```
id UUID PRIMARY KEY
artifact_id UUID NOT NULL
artifact_version_id UUID NOT NULL
project_id UUID NOT NULL
source_type VARCHAR(64) NOT NULL
source_id UUID
operation VARCHAR(128) NOT NULL
input_artifacts JSONB
input_versions JSONB
model_version_id UUID
revision_id UUID
code_manifest_id UUID
validation_run_id UUID
job_id UUID
actor_id UUID
metadata JSONB
created_at TIMESTAMPTZ NOT NULL
```

28. GENERATED ARTIFACT PROVENANCE

A drawing generated by OneDraft must be traceable through:

```
DRAWING ARTIFACT
↓
DRAWING RECORD
↓
VIEW
↓
MODEL VERSION
↓
REVISION
↓
PROJECT MODEL
```

If compliance information influenced generation:

CODE MANIFEST

↓

VALIDATION

↓

DRAWING

29. FINAL DOCUMENT PROVENANCE

A final PDF package must be traceable through:

DOCUMENT PACKAGE

↓

REVISION

↓

MODEL VERSION

↓

CODE MANIFEST

↓

VALIDATION RUN

↓

DRAWING SET

↓

ARTIFACT VERSIONS

This chain is mandatory for finalized project packages.

30. IMMUTABILITY RULE

Once an artifact version becomes:

FINAL

or:

ACCEPTED

its content must never be modified in place.

Any change produces:

NEW ARTIFACT VERSION

and, where appropriate:

NEW REVISION

31. ARTIFACT STATUS

Initial statuses:

PENDING
ACTIVE
SUPERSEDED
ARCHIVED
DELETED
FINAL
ACCEPTED

A DELETED artifact retains its metadata where required for provenance and audit.

The physical object may be subject to lifecycle-management policy rather than immediate deletion.

32. RETENTION POLICY

Retention shall be configurable by artifact class.

Initial classes:

TEMPORARY
PROJECT
HISTORICAL
FINAL
REGULATORY
AUDIT

Final and regulatory artifacts should have retention rules that prevent accidental destruction.

33. TEMPORARY ARTIFACTS

Temporary artifacts may include:

intermediate renders
worker scratch files
temporary exports
preview images
transient conversion files

These may be automatically removed after a configured retention period.

Temporary artifacts must never be used as the authoritative source for a finalized deliverable.

34. ARTIFACT CLEANUP

Cleanup operations shall be asynchronous.

Conceptually:

```
IDENTIFY EXPIRED ARTIFACT
↓
VERIFY NOT REFERENCED
↓
VERIFY NOT FINAL
↓
VERIFY RETENTION POLICY
↓
DELETE OBJECT
↓
RECORD CLEANUP EVENT
```

The cleanup worker must never delete an artifact solely because it appears unused in one table.

35. STORAGE INTEGRITY CHECK

The system shall periodically verify stored artifacts.

Conceptually:

```
ARTIFACT VERSION
↓
READ OBJECT
↓
HASH CONTENT
↓
COMPARE STORED HASH
```

If the hashes differ:

INTEGRITY_FAILURE

The system shall preserve the metadata and create an audit event.

36. ARTIFACT CHECKSUM API

Endpoint:

GET /api/v1/projects/{project_id}/artifacts/{artifact_id}/versions/{version_id}/
integrity

Response:


```
{
  "data": {
    "artifact_version_id": "uuid",
    "algorithm": "SHA-256",
    "stored_hash": "...",
    "verified_hash": "...",
    "status": "VALID"
  }
}
```

37. ARTIFACT SEARCH

Endpoint:

GET /api/v1/projects/{project_id}/artifacts

Supported filters:

type
status
mime_type
created_after
created_before
updated_after
name
revision_id
model_version_id

Pagination follows the established OneDraft API standard.

38. ARTIFACT VERSION HISTORY

Endpoint:

GET /api/v1/projects/{project_id}/artifacts/{artifact_id}/versions

Response:

```
{
  "data": [
    {
      "version_id": "uuid",
      "version": 1,
      "hash": "...",
      "size": 102400,
      "created_at": "ISO-8601"
    }
  ]
}
```

Historical versions remain addressable.

39. ARTIFACT REFERENCE

Internal services should reference artifacts using:

```
artifact_id  
artifact_version_id
```

rather than raw object-storage URLs.

This prevents infrastructure details from leaking into domain contracts.

40. SIGNED URL SECURITY

Signed URLs shall:

expire

and:

be scoped to a specific object

and:

use the minimum required permissions

The initial expiration period should be configurable.

The application must never persist long-lived public object URLs for private project artifacts.

41. ACCESS CONTROL

Artifact access requires:

```
authenticated identity  
+  
organization membership  
+  
project authorization
```

Additional restrictions may apply to:

```
professional-review artifacts  
regulatory artifacts  
acceptance artifacts
```

42. ARTIFACT OWNERSHIP

The artifact service shall inherit project tenancy from:

project_id

The organization remains the upper-level ownership boundary.

Cross-project artifact access is prohibited unless explicitly authorized through a future controlled sharing mechanism.

43. ARTIFACT SHARING

Future versions may support:

temporary_share
professional_review_share
customer_share
external_authority_share

The MVP does not require permanent public artifact URLs.

44. FILE TYPE VALIDATION

Uploaded artifacts shall be validated against:

filename
declared MIME type
detected MIME type
file signature
extension
size

The declared MIME type shall not be trusted as the sole source of file classification.

45. ARTIFACT SECURITY SCAN

The upload pipeline shall support an asynchronous security scanning stage.

Conceptually:

UPLOAD
↓
HASH
↓
FILE TYPE VALIDATION
↓
SECURITY SCAN
↓
ACCEPT

A failed scan results in:

REJECTED

and no artifact version becomes active.

46. CONVERSION PIPELINE

Artifact conversion shall execute through workers.

Examples:

DWG → preview
DXF → normalized geometry
PDF → page images
PDF → extracted text
IMAGE → thumbnail
DOCX → PDF
MODEL → lightweight viewer format

The conversion result becomes a derivative artifact.

47. CONVERSION JOB PROVENANCE

Every conversion records:

source_artifact_version
converter
converter_version
parameters
job_id
output_artifact_version
timestamp

This ensures that generated derivatives remain reproducible.

48. MODEL ARTIFACTS

Computational model exports shall be treated as artifacts.

Examples:

```
project_model.json
geometry_bundle
IFC
DXF
DWG
OBJ
glTF
```

The authoritative model remains the OneDraft project model.

Export formats are representations of that model.

49. CAD EXPORT

CAD exports shall identify:

```
project_id
model_version_id
revision_id
export_format
generator_version
generation_job_id
content_hash
```

A CAD export therefore remains traceable to the computational model from which it was produced.

50. DRAWING ARTIFACTS

A generated drawing shall associate:

```
drawing_id
sheet_id
view_id
project_id
revision_id
model_version_id
artifact_version_id
```

The physical PDF, SVG, DXF or raster output is therefore derived documentation.

51. RENDER ARTIFACTS

Rendered images shall identify:

source_view_id
model_version_id
render_settings
camera
lighting
renderer
renderer_version
job_id

Rendering settings become part of provenance.

52. CODE ARTIFACTS

Regulatory source documents shall be stored as controlled artifacts where licensing and deployment policy permit.

The associated:

code_sources

record remains the regulatory-domain record.

The artifact stores the retrieved source material.

53. CODE SOURCE INTEGRITY

Each stored regulatory artifact shall have:

source_uri
retrieval_timestamp
content_hash
jurisdiction
authority
version
effective_date

This permits OneDraft to determine exactly which source material informed a code manifest.

54. ACCEPTED PACKAGE

When customer acceptance occurs:

DOCUMENT PACKAGE
↓
PACKAGE HASH
↓
ACCEPTANCE RECORD

The acceptance record must identify the exact package artifact version.

Acceptance therefore cannot ambiguously refer to a later regenerated package.

55. PACKAGE HASH

The package hash shall be calculated from the deterministic package manifest.

Conceptually:

SORTED ARTIFACT MANIFEST
↓
CANONICAL REPRESENTATION
↓
SHA-256
↓
PACKAGE_HASH

The package hash provides a stable identity for the exact deliverable set.

56. REPRODUCIBILITY

OneDraft shall support the following reconstruction operation:

PROJECT_ID
REVISION_ID
MODEL_VERSION_ID
CODE_MANIFEST_ID
VALIDATION_RUN_ID
↓
REGENERATE
↓
EXPECTED ARTIFACT MANIFEST

The resulting artifact hashes can then be compared against the historical package.

57. ARTIFACT RECONSTRUCTION TEST

The system must eventually pass:

Retrieve historical project revision

↓
Retrieve model version
↓
Retrieve code manifest
↓
Retrieve validation results
↓
Retrieve artifact manifest
↓
Regenerate documentation
↓
Compare artifact structure
↓
Compare content hashes where deterministic

This becomes a major OneDraft provenance test.

58. STORAGE TRANSACTION BOUNDARY

Artifact metadata and object content are separate systems.

Therefore:

DATABASE TRANSACTION

and:

OBJECT STORAGE OPERATION

cannot always participate in one native transaction.

The system shall therefore use explicit artifact lifecycle states.

Example:

PENDING
↓
OBJECT_UPLOADED
↓
VERIFIED
↓
COMMITTED

If database finalization fails, reconciliation can safely identify the orphaned object.

59. STORAGE RECONCILIATION

A periodic reconciliation worker shall compare:

DATABASE ARTIFACT RECORDS

against:

OBJECT STORAGE OBJECTS

It shall identify:

missing objects
orphaned objects
hash mismatches
unexpected objects
incomplete uploads

No automatic destructive correction shall occur without lifecycle-policy verification.

60. ORPHAN MANAGEMENT

An object is considered potentially orphaned when:

object exists
AND
no committed artifact version references it

The system shall initially mark it:

ORPHAN_CANDIDATE

before any deletion occurs.

61. STORAGE EVENTS

The artifact engine shall emit events such as:

ARTIFACT_CREATED
ARTIFACT_UPLOADED
ARTIFACT_VERIFIED
ARTIFACT_VERSION_CREATED
ARTIFACT_DERIVED
ARTIFACT_FINALIZED
ARTIFACT_ACCEPTED
ARTIFACT_SUPERSEDED
ARTIFACT_ARCHIVED
ARTIFACT_DELETED
ARTIFACT_INTEGRITY_FAILURE
ARTIFACT_SECURITY_FAILURE

These events integrate with the existing audit and runtime orchestration architecture.

62. EVENT PAYLOAD

Example:

```
{
  "event_type": "ARTIFACT_VERSION_CREATED",
  "project_id": "uuid",
  "artifact_id": "uuid",
  "artifact_version_id": "uuid",
  "content_hash": "sha256",
  "actor_id": "uuid",
  "timestamp": "ISO-8601"
}
```

The event is append-only.

63. JOB INTEGRATION

Artifact operations requiring substantial processing shall use the existing Job & Worker Execution System.

Examples:

```
SECURITY_SCAN
FILE_CONVERSION
TEXT_EXTRACTION
PDF_RENDERING
CAD_EXPORT
DOCUMENT_PACKAGING
HASH_VERIFICATION
STORAGE_RECONCILIATION
```

The artifact engine shall submit jobs rather than blocking API requests.

64. WORKER INTERFACE

Workers shall consume commands conceptually such as:

```
artifact.verify
artifact.convert
artifact.package
artifact.hash
artifact.scan
artifact.reconcile
artifact.cleanup
```

The existing runtime orchestrator remains responsible for scheduling and execution control.

65. ARIA INTEGRATION

Aria shall not directly manipulate object storage.

Aria may request capabilities such as:

```
create_artifact
retrieve_artifact
create_derivative
package_artifacts
export_model
generate_document
```

The artifact service performs the actual storage operation.

The AI boundary therefore remains:

```
ARIA
↓
DOMAIN CAPABILITY
↓
ARTIFACT SERVICE
↓
STORAGE
```

66. AI-GENERATED ARTIFACTS

Artifacts generated through Aria must identify:

```
aria_command_id
generation_job_id
model_version_id
revision_id
```

This allows every AI-generated deliverable to be traced back to the operation that produced it.

67. ARTIFACT GENERATION SAFETY

Aria cannot directly:

```
overwrite final artifacts
delete historical artifacts
alter accepted packages
modify regulatory source artifacts
change retention policies
change storage credentials
```

These operations remain application-controlled capabilities.

68. API RESOURCE GROUPS

The artifact API shall initially expose:

- Artifacts
- Artifact Versions
- Uploads
- Downloads
- Derivatives
- Packages
- Integrity
- Storage Jobs

69. API ENDPOINTS

Initial endpoints:

- POST /api/v1/projects/{project_id}/artifacts/uploads
- POST /api/v1/projects/{project_id}/artifacts/uploads/{upload_id}/complete
- GET /api/v1/projects/{project_id}/artifacts
- GET /api/v1/projects/{project_id}/artifacts/{artifact_id}
- GET /api/v1/projects/{project_id}/artifacts/{artifact_id}/versions
- GET /api/v1/projects/{project_id}/artifacts/{artifact_id}/download
- GET /api/v1/projects/{project_id}/artifacts/{artifact_id}/integrity
- GET /api/v1/projects/{project_id}/packages
- GET /api/v1/projects/{project_id}/packages/{package_id}

70. ARTIFACT CREATION RESPONSE

```
{
  "data": {
    "artifact_id": "uuid",
    "artifact_version_id": "uuid",
    "status": "ACTIVE",
    "content_hash": "sha256",
    "size": 18420342
  },
  "metadata": {
    "request_id": "uuid",
    "timestamp": "ISO-8601"
  }
}
```

71. PACKAGE RETRIEVAL RESPONSE

```
{
  "data": {
    "package_id": "uuid",
    "project_id": "uuid",
    "revision_id": "uuid",
    "model_version_id": "uuid",
    "code_manifest_id": "uuid",
    "validation_run_id": "uuid",
    "package_hash": "sha256",
    "status": "FINAL",
    "artifact_count": 37
  }
}
```

72. DATABASE INDEX STRATEGY

Primary artifact indexes:

```
project_id
artifact_id
artifact_type
status
created_at
updated_at
content_hash
artifact_version_id
document_package_id
revision_id
model_version_id
job_id
```

The database shall be profiled before adding speculative indexes.

73. DATABASE MIGRATION ORDER

The initial migration sequence shall extend the existing OneDraft migrations:

```
026_storage
027_artifacts
028_artifact_versions
029_uploads
030_derivatives
031_artifact_links
032_packages
033_package_items
```

034_provenance
035_storage_indexes
036_storage_events
037_storage_seed_data

Each migration is immutable once applied.

74. CONFIGURATION

Initial configuration values shall include:

STORAGE_PROVIDER
STORAGE_BUCKET
STORAGE_REGION
STORAGE_ENDPOINT
STORAGE_ACCESS_KEY
STORAGE_SECRET_KEY
SIGNED_URL_EXPIRATION
MAX_UPLOAD_SIZE
ARTIFACT_RETENTION_DAYS
TEMPORARY_ARTIFACT_RETENTION_DAYS
HASH_ALGORITHM

Secrets must be supplied through the deployment secret-management mechanism.

They must never be committed to source control.

75. LOCAL DEVELOPMENT

The development stack should support:

PostgreSQL
+
S3-compatible object storage
+
OneDraft API
+
Worker

A local S3-compatible provider allows development without coupling the development environment to a commercial cloud storage provider.

76. TEST STORAGE

Development artifacts shall use clearly separated storage:

onedraft-dev
onedraft-test
onedraft-production

Production artifacts must never be written to development storage.

77. ARTIFACT TEST SUITE

The initial artifact test suite shall verify:

artifact creation
upload initialization
upload completion
hash verification
artifact retrieval
artifact download authorization
artifact version creation
derivative creation
package creation
package manifest generation
package hashing
artifact provenance
integrity verification
orphan detection
retention handling
access control

78. INTEGRATION TEST

The first complete storage integration test shall execute:

1. Create project
2. Initialize artifact upload
3. Upload source document
4. Complete upload
5. Verify content hash
6. Create artifact version
7. Generate derivative
8. Record provenance
9. Create project model
10. Generate drawing

11. Store drawing artifact
12. Generate document package
13. Store package
14. Generate package manifest
15. Calculate package hash
16. Record validation relationship
17. Record customer acceptance
18. Retrieve package
19. Verify package hash
20. Reconstruct artifact lineage

This becomes the first major Artifact Engine regression test.

79. FAILURE RECOVERY

The artifact subsystem shall tolerate failures including:

- interrupted upload
- worker failure
- object storage timeout
- database timeout
- hash mismatch
- conversion failure
- security scan failure
- package generation failure
- temporary storage outage

The system shall preserve enough state to retry safely.

80. IDEMPOTENCY

Mutating artifact operations shall support:

Idempotency-Key

Repeated requests must not create duplicate artifact versions when the same logical operation has already completed.

81. CONCURRENCY CONTROL

Artifact version creation shall use transactional concurrency protection.

The system must prevent two concurrent writers from incorrectly assigning the same:

`artifact.version_number`

Optimistic concurrency should be preferred where appropriate.

82. FINALIZATION BOUNDARY

An artifact may transition to:

FINAL

only when the owning domain workflow authorizes finalization.

For document packages this requires, where applicable:

`valid project revision`
`valid model version`
`valid code manifest`
`completed validation`
`completed document generation`
`complete package manifest`
`successful package hashing`

83. ACCEPTANCE BOUNDARY

An artifact package becomes:

ACCEPTED

only through the existing acceptance workflow.

Storage state alone cannot create customer acceptance.

84. SYSTEM OF RECORD

The OneDraft hierarchy now becomes:

PROJECT MODEL
↓
REVISION HISTORY
↓

REGULATORY MANIFEST
↓
VALIDATION RESULTS
↓
GENERATED DOCUMENTATION
↓
ARTIFACT VERSIONS
↓
DOCUMENT PACKAGE
↓
CUSTOMER ACCEPTANCE
↓
AUDIT EVENTS

The artifact layer preserves the physical representation of the derived project state.

85. RECONSTRUCTION CHAIN

A completed project must support:

PROJECT_ID
↓
REVISION_ID
↓
MODEL_VERSION_ID
↓
CODE_MANIFEST_ID
↓
VALIDATION_RUN_ID
↓
DOCUMENT_PACKAGE_ID
↓
ARTIFACT_MANIFEST
↓
ARTIFACT_VERSIONS
↓
OBJECT_STORAGE_CONTENT

This provides end-to-end provenance.

86. ARCHITECTURAL BOUNDARY

OneDraft shall not be implemented as:

DATABASE
+
RANDOM FILE STORAGE
+
PDF EXPORT

It shall instead operate as:

COMPUTATIONAL PROJECT MODEL
+
VERSION CONTROL
+
REGULATORY STATE
+
VALIDATION
+
DOCUMENTATION
+
ARTIFACT MANAGEMENT
+
OBJECT STORAGE
+
PROVENANCE

The artifact is therefore a controlled representation of project state, not an orphaned file.

87. MVP ARTIFACT ENGINE SUCCESS CRITERIA

The subsystem is successful when it can:

Store project artifacts

Version artifact content

Calculate cryptographic hashes

Support direct uploads

Generate secure downloads

Store derivatives

Track artifact relationships

Track provenance

Create document packages

Generate package manifests

Verify package integrity

Integrate with workers

Integrate with revisions

Integrate with validation

Integrate with documentation

Record acceptance references

Recover from interrupted operations

Reconstruct artifact lineage

88. EXECUTABLE FOUNDATION ARTIFACTS

The next implementation package shall consist of:

Artifact A

`026_storage.sql`

The initial storage metadata migration.

Artifact B

`artifact_service.py`

The application service controlling artifact lifecycle.

Artifact C

`storage_provider.py`

The provider-neutral object-storage interface.

Artifact D

`s3_storage_provider.py`

The initial S3-compatible implementation.

Artifact E

`artifact_models.py`

Python/Pydantic artifact-domain models.

Artifact F

`artifact_worker.py`

The initial worker handlers for verification, hashing and derivative processing.

Artifact G

artifact_routes.py

The REST API implementation.

Together these artifacts establish the executable OneDraft storage foundation.

89. INTEGRATION WITH THE EXISTING STACK

The Artifact & Storage Management Engine does not replace any established subsystem.

It attaches to:

```
Project Model
  ↓
Geometry Engine
  ↓
Compliance Engine
  ↓
Validation Engine
  ↓
Revision Engine
  ↓
Documentation Engine
  ↓
Runtime Orchestrator
  ↓
Job & Worker Execution
  ↓
Artifact & Storage Engine
```

The storage engine therefore becomes the persistent physical-output boundary of the existing OneDraft architecture.

90. VERSION 0.1 ENGINEERING STATE

The OneDraft architecture now contains:

Product Definition

System Architecture

Technology Architecture

Domain Model

Persistence Model

OpenAPI Contract

Geometry & Parametric Model Engine

Compliance & Regulatory Engine

Validation Engine

Revision & Redline Engine

Documentation & Drawing Generation Engine

Runtime Orchestrator

Job & Worker Execution System

Artifact & Storage Management Engine

The architecture now possesses both:

COMPUTATIONAL STATE

and:

PHYSICAL ARTIFACT STATE

with a controlled provenance relationship between them.

91. NEXT IMPLEMENTATION STAGE

The next engineering stage proceeds from artifact persistence into the **OneDraft Integration & Event Bus**, where the established subsystems become coordinated through typed domain events and inter-service contracts.

The next stack shall establish:

DOMAIN EVENTS

+

EVENT CONTRACTS

+

EVENT BUS

+

OUTBOX PATTERN

+

EVENT CONSUMERS

+

WORKFLOW TRIGGERS

+

CROSS-SUBSYSTEM COORDINATION

+

EVENT REPLAY

+

FAILURE RECOVERY

This layer will connect the existing:

PROJECT
GEOMETRY
COMPLIANCE
VALIDATION
REVISION
DOCUMENTATION
RUNTIME
JOB
ARTIFACT

subsystems into one executable OneDraft platform.

92. SPECIFICATION STATUS

ONEDRAFT ARTIFACT & STORAGE MANAGEMENT ENGINE v0.1

STATUS: ESTABLISHED

The next implementation stage is the creation of the executable artifact foundation:

`026_storage.sql`

`artifact_service.py`

`storage_provider.py`

`s3_storage_provider.py`

`artifact_models.py`

`artifact_worker.py`

`artifact_routes.py`

These artifacts constitute the physical persistence and provenance foundation for OneDraft-generated project deliverables.